Apuntes sobre Expresiones Regulares e Irregulares

1. Lenguaje natural y lenguaje formal

El **lenguaje natural** es el que usamos las personas para comunicarnos (español, inglés...), es flexible, ambiguo y cambia con el tiempo.

El **lenguaje formal** es el que usan las máquinas: tiene reglas muy estrictas, sin ambigüedad. Ahí entran los lenguajes de programación y también los patrones para buscar texto.

Las **expresiones regulares** son una forma de describir un conjunto de cadenas de texto con reglas formales, para que el ordenador pueda reconocerlas automáticamente.

2. ¿Qué es una expresión regular?

Una **expresión regular** (regex o regexp) es un patrón de búsqueda que sirve para encontrar, validar, reemplazar o extraer texto dentro de cadenas de caracteres.

En lugar de buscar una palabra exacta, definimos *qué forma* debe tener el texto (por ejemplo: "un correo electrónico", "un DNI", "5 dígitos seguidos...").

Se usan en muchos lenguajes: PHP, JavaScript, Python, Java, etc. Nos permiten buscar coincidencias sin tener que escribir muchos bucles.

3. ¿Expresión regular o irregular?

El término correcto en informática es **expresión regular**. La palabra "regular" hace referencia a las *lenguas regulares* de la teoría de autómatas (Noam Chomsky).

Las **lenguas regulares** son aquellas que pueden ser reconocidas por autómatas finitos, y las expresiones regulares fueron creadas para describir exactamente ese tipo de lenguajes formales.

Decimos "expresión regular", no "irregular". Cuando alguien dice "irregular", se confunde con los verbos irregulares del inglés/español.

Dato importante: Existe la *regex* o *expresión regular*. Nunca se dice "expresión irregular" en el ámbito informático.

4. Metacaracteres más usados

Los **metacaracteres** son símbolos con un significado especial dentro de una expresión regular.

Metacaracter	Significado	Ejemplo	Coincide con...
`.`	Cualquier carácter (excepto salto de línea)	`c.t`	"cat", "cut", "c3t"
`^`	Inicio de cadena	`^Hola`	Solo si empieza por "Hola"
`$`	Final de cadena	`mundo$`	Solo si termina por "mundo"

Metacaracter	Significado	Ejemplo	Coincide con...
`|`	O (alternancia)	`perro|gato`	"perro" o "gato"
`[]`	Clase de caracteres	`[aeiou]`	Cualquier vocal
`[^ ]`	Negación de clase	`[^0-9]`	Cualquier cosa que NO sea un dígito
`()`	Grupo de captura	`(abc)+`	Captura "abc" para reutilizarlo
`\`	Escape de metacaracter	`\.`	Busca un punto literal "."

5. Cuantificadores

Indican cuántas veces debe aparecer un carácter, grupo o clase.

Cuantificador	Significado	Ejemplo	Coincide con...
`*`	0 o más veces	`a*`	"", "a", "aaa"
`+`	1 o más veces	`a+`	"a", "aaa" (no vacío)
`?`	0 o 1 vez (opcional)	`colou?r`	"color", "colour"
`{n}`	Exactamente n veces	`\d{3}`	3 dígitos
`{n,}`	n o más veces	`\d{2,}`	2 o más dígitos

Cuantificador	Significado	Ejemplo	Coincide con...
`{n,m}`	Entre n y m veces	`\d{1,3}`	1, 2 o 3 dígitos

6. Secuencias predefinidas

Atajos muy útiles para clases comunes. Todas llevan una barra invertida.

Símbolo	Equivale a	Descripción
`\d`	`[0-9]`	Dígito
`\D`	`[^0-9]`	No dígito
`\w`	`[a-zA-Z0-9_]`	Carácter alfanumérico o guión bajo
`\W`	`[^a-zA-Z0-9_]`	No alfanumérico
`\s`	`[ \t\r\n\f\v]`	Espacio en blanco
`\S`	`[^ \t\r\n\f\v]`	No espacio en blanco

7. Ejemplos prácticos

Validar un número de teléfono español (9 dígitos)

```
/^\d{9}$/
```

`^` inicio, `\d{9}` 9 dígitos, `$` fin. Ej: "123456789" válido.

Validar un correo electrónico básico

```
/^[\w.-]+@[\w.-]+\.[a-zA-Z]{2,}$/
```

Busca letras/números/puntos/guiones, @, dominio y extensión de 2 o más letras.

Extraer palabras que empiecen por mayúscula

```
/\b[A-Z][a-z]*\b/
```

`\b` límite de palabra. Ej: "Ana y Pedro" extrae Ana, Pedro.

Buscar solo fechas con formato DD/MM/AAAA

```
/^\d{2}\/\d{2}\/\d{4}$/
```

La barra se escapa con `\`. Ej: "05/11/2025" válido.

Sustituir varios espacios por uno solo

```
/\s+/
```

Reemplazando por " " deja una sola separación entre palabras.

8. Expresiones "regulares" vs "irregulares" (aclaración final)

Concepto	Correcto	Explicación
Patrones de texto	**Expresión regular**	Término estándar en programación e informática.
Verbos en inglés	**Verbo irregular**	No sigue la regla "-ed" para pasado (go → went).

Moraleja: Cuando hablamos de búsquedas con patrones, decimos "*expresión regular*". La forma "*expresión irregular*" es incorrecta en este contexto.

9. Breve cronología y uso a lo largo del tiempo

> **1950–1956** – Stephen Cole Kleene introduce el concepto de *expresiones regulares* dentro de la teoría de autómatas y lenguajes formales. Su objetivo era describir lenguas regulares de forma matemática.
>
> **1968** – Ken Thompson las implementa por primera vez en UNIX, concretamente en el editor *ed* y en la herramienta *grep* (de *Global Regular Expression Print*). Ahí empiezan a salir de la teoría para usarse en la práctica.
>
> **Década de 1970–1980** – Se extienden por UNIX (sed, awk, grep con variantes). Se popularizan en sistemas operativos y herramientas de texto.
>
> **Década de 1990** – Llegan a los lenguajes de programación (Perl, Tcl, Python, JavaScript, Java...). Perl fue clave para estandarizar y ampliar su uso cotidiano.
>
> **Década de 2000–actualidad** – Se usan en casi todos los lenguajes, editores, IDEs, buscadores, validaciones de formularios web, análisis de logs, extracción de datos, expresiones para rutas, seguridad (detección de patrones...), IA, automatizaciones, etc.

¿Para qué se usan hoy en día?

Las expresiones regulares tienen un uso muy práctico hoy en día:

- **Validación de formularios:** comprobar correos, teléfonos, DNIs, contraseñas...
- **Búsqueda y reemplazo:** encontrar y cambiar texto en miles de líneas de golpe.
- **Procesamiento de datos:** limpiar, extraer o filtrar información (logs, CSV, JSON...).
- **Automatización:** scripts que filtran archivos, nombres o rutas.
- **Seguridad y análisis:** detectar patrones sospechosos o filtrar entradas.
- **Desarrollo web:** validaciones en cliente (JavaScript) y servidor (PHP, Node...), rutas, etc.

Conclusión: Nacieron para resolver problemas teóricos (lenguajes regulares), pero hoy son una herramienta cotidiana para trabajar con texto de forma rápida, precisa y eficiente.